UNIFAAT 2026.2

05 - Git & GitHub na prática

Prof. Luan Lourenço

Revisão rápida

Um arquivo editado duas vezes no VS Code, dois commits, e um `reset --hard` pra reaquecer — o mesmo fluxo da aula passada, de novo com as mãos no teclado.

recap-aula-04/

```
# cria o repositório .git nesta pasta
$ git init
# manda os arquivos pra área de preparação (staging)
$ git add .
# grava um snapshot permanente do que foi preparado
$ git commit -m "Cria anotacoes.txt..."
$ git add .
$ git commit -m "Adiciona segunda linha..."
# --oneline resume cada commit em uma linha só
$ git log --oneline
# desfaz o último commit, mesmo ele já tendo sido salvo
$ git reset --hard HEAD~1
```

Todo esse histórico, até agora, vive só nessa máquina. É esse ponto que a aula de hoje resolve.

cmd → wsl → Desktop

> ! O path da Área de Trabalho muda de PC pra PC — `whoami` no WSL não ajuda, porque o usuário do Linux não é o do Windows. Prefere ver visualmente? Abre o Explorador de Arquivos com `⊞ Win` + `E` e navega até a Área de Trabalho — o caminho aparece na barra de endereço lá em cima.

cmd.exe

```
# entra no ambiente Linux (WSL) a partir do cmd
C:\> wsl
# descobre o nome de usuario certo neste PC
$ ls /mnt/c/Users/
# entra na Área de Trabalho desse usuário
$ cd /mnt/c/Users/NOME/Desktop
```

Baixando o `.git` pronto

O material desta aula já é um único arquivo `.zip`, com a pasta `.git` pronta lá dentro, disponível num link do Google Drive.

- → Abra o link no navegador (pode ser no Windows mesmo, fora do WSL): `drive.google.com/file/d/1LgCeesIrCrYMg-vnU2Hn95rlCTEcH5Ey/view`
- → Clica no ícone de **download** (a setinha, no canto superior direito da prévia do arquivo)
- → O navegador baixa o `.zip` — por padrão, ele cai na pasta **Downloads**, não na Área de Trabalho

Pra extrair o `.zip`, tanto faz usar o terminal do WSL ou o Explorador de Arquivos do Windows — escolham o que for mais confortável:

Desktop/

```
# pelo WSL: cria e extrai tudo numa pasta nova
# ajuste NOME ao nome real do arquivo
$ unzip /mnt/c/Users/NOME/Downloads/*.zip \
    -d es-aula-05
$ cd es-aula-05
```

> **Pelo Explorador de Arquivos:** vai em **Downloads**, botão direito no `.zip` → **Extrair tudo...**, escolhe a Área de Trabalho como destino e renomeia a pasta pra `es-aula-05`.

Pelo WSL, não precisou de `mkdir` — o `-d` do `unzip` já cria a pasta `es-aula-05` sozinho. É o mesmo resultado dos dois jeitos: uma pasta `es-aula-05` na Área de Trabalho, com o `.git` lá dentro.

atenção

Vocês vão receber só isto:

es-aula-05/

```
# -a mostra também os arquivos ocultos (que começam com .)
$ ls -la
.git
```

Nenhum `index.html`. Nenhum `css/`. Só a pasta oculta. O projeto inteiro vai "aparecer" a partir dela.

! Não vê a pasta `.git` no Explorador de Arquivos nem no VS Code? Normal — pastas com `.` na frente ficam ocultas por padrão. No VS Code: `Ctrl` + `,` → busca `files.exclude` → desmarca `**/.git`.

Some. E volta.

es-aula-05/

```
# mostra o que mudou em relação ao último commit
$ git status
deleted:    .gitignore
deleted:    aula-05-git-github/briefing-aula-05.md
deleted:    aula-05-git-github/roteiro-aula-05.md
deleted:    aula-05-git-github/slide/index.html
# (+ 5 outros arquivos, todos dentro de aula-05-git-github/)
# traz de volta os arquivos apagados, usando o que já foi commitado
$ git restore .
$ ls -la
.gitignore  aula-05-git-github
```

"Deletados"? Vocês nunca chegaram a ter esses arquivos — e um único comando trouxe tudo de volta. Reparem: é o próprio material desta disciplina que está sendo reconstruído.

Só o que foi commitado mora dentro do `.git` — e existe um jeito de garantir que um arquivo nunca chegue a ser commitado. Isso é assunto do próximo slide.

`.gitignore`: o que nunca chega a existir

`.gitignore` é um arquivo de texto comum, na raiz do projeto, com uma lista de padrões de nomes de arquivo. Tudo que casa com esses padrões o Git passa a ignorar por completo — nunca aparece no `git status`, nunca entra num `git add`, nunca vira commit.

es-aula-05/

```
# o .gitignore deste próprio repositório da disciplina
$ cat .gitignore
*.docx
```

Um exemplo real: o briefing desta aula também existe como `briefing-aula-05.docx`, gerado pra facilitar a revisão fora do terminal. Mas como `*.docx` está no `.gitignore`, esse arquivo nunca foi commitado — ele mora só no disco, nunca dentro do `.git`.

Por isso não existe um "restaurar o .docx": ele nunca teve um snapshot salvo. A lição do dia, em uma frase: **só o que foi commitado mora dentro do `.git`** — o resto é só arquivo solto no disco, ignorado de propósito ou não.

`HEAD` e a branch: ponteiro pra ponteiro

es-aula-05/

```
# mostra pra qual branch o HEAD está apontando agora
$ cat .git/HEAD
ref: refs/heads/main
# mostra o hash do commit que a branch main aponta
$ cat .git/refs/heads/main
1878ee023677560b204d2427f3a3c900c1c0986b
```

`HEAD` não guarda um commit direto — aponta pra **outro arquivo** (a branch), que aponta pro commit.

Mesmo hash do topo do `git log`. Uma branch é só um arquivo de texto com um hash dentro.

`config` — antes do GitHub

es-aula-05/

```
# mostra as configurações locais deste repositório
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
```

Guarda esse arquivo na cabeça — ele ganha uma seção nova daqui a pouco.

Repositório: o lugar, não o conteúdo

Repositório, no sentido amplo da palavra, é só isso: um lugar onde algo fica guardado, armazenado — nem todo repositório é de código. Um repositório de arquivos, de dados, de peças — a palavra não é exclusiva do Git.

No universo do Git, o que se guarda dentro desse lugar é justamente o `.git` — o histórico completo de versões de um projeto (`objects`, `refs`, `HEAD`). Até aqui, o "lugar" onde esse `.git` está guardado é uma única pasta, numa única máquina.

```
# repositório local
o lugar: seu HD.
o que ele guarda: o .git.
```

```
# repositório no GitHub
o lugar: um servidor na internet.
o que ele guarda: o mesmo .git.
```

GitHub é um serviço de hospedagem — ele oferece o "lugar" na internet. O que fica guardado ali dentro é exatamente o `.git`, por isso `clone`, `push` e `pull` conseguem trafegar esse conteúdo entre sua máquina e o GitHub, do mesmo jeito que trafegariam entre duas máquinas quaisquer.

Criando o repositório no GitHub

- Em `github.com`, clica no `+` (canto superior direito) → `New repository`
- Dá um nome e escolhe `Public` ou `Private`
- Não marca `Add a README` — o repo tem que nascer vazio
- Confirma em `Create repository`
- Na página do repo recém-criado: botão verde `Code` → aba `SSH` → copia a URL

es-aula-05/

```
# cadastra o endereço remoto com o apelido "origin"
$ git remote add origin git@github.com:usuario/repo.git
# confirma que o endereço foi salvo no config
$ cat .git/config
[core] ...
[remote "origin"]
    url = git@github.com:usuario/repo.git
```

`git remote add` não fez mágica de rede — só escreveu duas linhas num arquivo texto.

Chave SSH: gerando e cadastrando

A pública é o cadeado que vocês entregam pro GitHub guardar. A privada é a única chave que abre — e fica só com vocês, nunca sai da máquina.

~

```
# gera o par de chaves SSH (pública e privada)
$ ssh-keygen -t ed25519 -C "voce@email.com"
# mostra a chave pública, pra copiar
$ cat ~/.ssh/id_ed25519.pub
# testa se o GitHub reconhece a chave
$ ssh -T git@github.com
Hi usuario! You've successfully authenticated...
```

- → Clica na sua **foto de perfil** (canto superior direito) → `Settings`
- → No menu lateral, entra em `SSH and GPG keys`
- → Botão verde `New SSH key`
- → Em `Title`, dá um nome pra identificar essa máquina (ex: "meu notebook")
- → Em `Key`, cola o conteúdo copiado do `id_ed25519.pub`
- → Confirma em `Add SSH key`

Vendo essa mensagem, a autenticação já está pronta — o próximo `git push` não vai pedir senha nenhuma.

`clone`, `push` e `pull`

```
# baixar (1ª vez)
git clone <url>
```

```
# enviar — flexão
git push -u origin main
```

```
# receber — barra
git pull
```

`git clone` é o comando de "primeira vez": baixa o repositório inteiro do GitHub (todo o `.git`) e já cria a pasta local, sem precisar de `git init` nem `git remote add` na mão.

Decompondo `git push -u origin main`: `origin` é o **remote** (pra onde vai), `main` é o **branch** (o que vai), e `-u` marca os dois como o padrão — só precisa na primeira vez. Depois disso, `git push` e `git pull` sozinhos já sabem pra onde ir.

Pensa em calistenia: `push` é **flexão** — você empurra o peso (o código) pra longe do corpo, pro GitHub. `pull` é **barra** — você puxa o peso pra perto, do GitHub pra sua máquina.

`clone` é usado uma vez, quando o repositório local ainda não existe. `push` e `pull` são o dia a dia depois disso: flexão pra empurrar o que é seu pra fora, barra pra puxar o que tá lá fora pra perto de você.

SO, terminal, WSL, Git, GitHub.

A base da primeira parte do ano está fechada. A seguir: organizar o que se constrói — Python e orientação a objetos.

tarefa

TF 05: Git do zero até o GitHub

Criar um repositório novo, garantir que um arquivo secreto nunca seja commitado, e subir tudo pra um repositório no GitHub.

- → `git init` num diretório novo (`tf-05`)
- → `segredo.txt` na raiz, listado no `.gitignore` antes do primeiro commit
- → Pelo menos 2 commits com arquivos reais (`sobre-mim.txt`, `anotacoes-aula-05.txt`)
- → Repositório criado no GitHub, conectado com `git remote add` e enviado com `git push`

Entrega: link do repositório no GitHub, enviado pelo formulário abaixo — `docs.google.com/forms/d/e/1FAIpQLSd-NZVE4P9hfsKsxjiwlJG33FCE5JyUjbcEubCpCP7lX1OEPA/viewform`

Prazo: **23/09, até 23:59**.